

PSF-Zero: An Analytic Two-Qubit Gate Synthesizer Combined with a Verified, Ordering-Aware Layout Search for Quantum Circuit Transpilation

TN Holdings

`love.os.architect@proton.me`

<https://github.com/TN-Holdings-LLC/psf-zero>

Version 2 — 26 September 2026

Abstract

We present PSF-Zero, a two-part quantum circuit compiler consisting of (1) an analytic two-qubit gate synthesizer based on the Cartan (KAK) decomposition, implemented as a compiled Rust core, and (2) a qubit layout search built around a specific ordering strategy shown effective in a companion paper. On a fixed 15-qubit, dense-pair-block circuit family, the gate synthesizer reproduces Qiskit’s own output to a fidelity of 1.000000000000 at small scale and achieves a $7.0\text{--}9.3\times$ cumulative speedup over Qiskit’s default `optimization_level=3` synthesis path across independently repeated 10,000-iteration measurements; we report the variance of this speedup honestly, including a $34\text{--}54\times$ reduction in timing variance observed between two measurement sessions whose cause we investigated with the actual source diff from both sessions and could not identify. The layout search, verified through the public `compile_for_hardware()` entry point rather than as an isolated component, finds a valid embedding on 26 of 26 saturated test instances—up from 0 of 26 before a feasibility-check defect we found and fixed—with all 26 outputs independently verified structurally valid (zero coupling-map violations) and, at small scale, unitary-equivalent to the input circuit to machine precision (minimum fidelity 0.999999999999994 across 36 independent checks). We further verify the same layout search on two topologies distinct from the square grids used in development, including one (`brick`) on which the search’s own earlier ordering strategies had previously found nothing at all. We report every defect found in the course of this verification, including one whose root cause we mischaracterized in an earlier stage of this investigation and later corrected, and one inconsistency in our own reported timing figures that we found and correct in this paper. All code, raw data, and the complete chronological verification record are public.

Revision note (version 2). Every result reported in version 1 stands and is reproduced here unchanged in Sections 2–6. Version 2 adds Section 7, which reports work done after version 1: a defect in the CX-basis synthesis that PSF-Zero inherited from Qiskit, found on 26 September 2026, together with its scope, why it does not affect the measurements of version 1, and its fix; the precision of PSF-Zero’s output under repeated recompilation; compile time on a current IBM topology; and an error-aware layout, evaluated on a model. Two sentences in Sections 2 are marked as added in version 2.

1 Introduction

Quantum circuit compilation for near-term hardware combines two largely independent problems: synthesizing an arbitrary computation into a device’s native gate set, and mapping the circuit’s logical qubits onto physical ones subject to connectivity constraints. PSF-Zero addresses both. Its gate synthesis path replaces Qiskit’s own general-purpose two-qubit decomposition with a specialized, analytic KAK/Cartan decomposition compiled to native code. Its layout path is a prototype search built around a specific empirical finding—that a particular, extremely simple node-ordering strategy resolves a class of instances on which Qiskit’s own production layout pass fails outright—reported in detail in a companion paper [1].

This paper’s contribution is not the ordering insight itself, which belongs to the companion paper; it is the engineering and verification of a system that uses it. We treat "the search finds a layout" as necessary but insufficient, and verify three further properties that, to our knowledge, are not jointly established for any comparable research prototype: that every found layout respects the device’s connectivity (structural validity), that the resulting circuit computes the same operator as the input (unitary equivalence, checked exactly at small scale rather than assumed), and that the underlying search strategy is not overfit to the square-grid topology it was developed on.

We are equally direct about a result we cannot explain. Over the course of this work we observed the gate synthesizer’s own timing variance drop by more than an order of magnitude between two measurement sessions using nominally the same benchmark. We obtained the actual source diff between the two sessions’ codebases, traced every candidate mechanism we could identify, ruled out each one by direct test, and did not find the cause. We report this as a closed, unresolved question rather than an unsupported explanation, consistent with the verification standard we apply to every positive claim in this paper.

1.1 Contributions

1. An analytic two-qubit gate synthesizer achieving $7.0\text{--}9.3\times$ cumulative speedup over Qiskit’s default synthesis path, with exact correctness verified at small scale (Section 2).
2. An engineered layout search, built on the ordering finding of a companion paper, verified end-to-end through the system’s own public API: 26/26 instance coverage (up from 0/26 before a defect fix), zero structural violations, and unitary equivalence to machine precision (Section 3).
3. Generalization evidence on two topologies outside the search’s own development set, including a topology on which its predecessor strategies previously found nothing (Section 4).
4. A full account of defects found during development and verification, including one root-cause mischaracterization we corrected mid-investigation and one figure-transcription error we correct in this paper (Section 5).
5. An honestly unresolved empirical finding—the gate synthesizer’s own variance reduction across sessions—reported with the investigation that failed to explain it (Section 6).
6. (*Version 2.*) A defect in Qiskit’s CX-basis two-qubit synthesis, its effect on PSF-Zero and its fix; precision under repeated recompilation; compile time on a current IBM topology; and an error-aware layout (Section 7).

2 Gate synthesis

2.1 Design

PSF-Zero’s `compile()` entry point collects two-qubit blocks from a circuit (via Qiskit’s own `Collect2qBlocks` and `ConsolidateBlocks` passes, reused rather than reimplemented) and replaces each consolidated block’s unitary with an explicit KAK/Cartan decomposition: two single-qubit unitaries, an entangling core expressed in the target basis (`cx` in all measurements reported here), and two further single-qubit unitaries. The decomposition itself runs in a compiled Rust extension (`psf_zero_core`), called once per consolidated block. (*Added in version 2.*) Up to version 1, the entangling core was re-expressed in the `cx` basis by Qiskit’s own two-qubit decomposer; Section 7.1 reports a defect in that decomposer and how PSF-Zero now avoids it.

2.2 Correctness

At small scale ($n = 6$), where exact operator comparison is tractable, the synthesized circuit reproduces the input circuit’s own unitary to a fidelity of 1.000000000000 across all three arms compared (Qiskit `optimization_level=3`, PSF-Zero with and without its own verification pass enabled). We do not compute exact unitary equivalence at the 15-qubit scale used for the timing measurements below; this is the same limitation stated plainly in our own project documentation, and we repeat it here rather than let the small-scale result imply more than it establishes. (*Added in version 2.*) The comparisons above are of the emitted circuit’s operator and are independent of PSF-Zero’s built-in check: `verify=True` validates the decomposition returned by the Rust core, not the circuit finally emitted, and so could not have caught the defect of Section 7.1.

2.3 Performance

Session	Total (Qiskit)	Total (PSF-Zero)	Speedup	Std. dev. (PSF-Zero)
Archive (4 days prior)	92.07 s	13.14 s	7.00×	0.902 ms
Today	103.75 s	11.86 s	8.75×	0.123 ms
Repeat	106.56 s	11.49 s	9.27×	0.164 ms

Table 1: Cumulative time over 10,000 iterations, 15-qubit dense-pair-block circuits, `verify=False`, three independent sessions. Standard deviation is per-call, over the same 10,000 calls. *The archive session’s speedup was previously reported as 8.87×* in our own project record; recomputed directly from the session’s own raw per-call data for this paper, it is 7.00×. We correct the earlier figure here (Section 5) rather than silently using the corrected value without comment.

We report a plain speedup range, 7.0–9.3×, rather than a single headline number, because the three independent sessions summarized in Table 1 do not agree to within measurement noise on this specific circuit family and hardware configuration. We consider the range itself, and the reasons behind it (Section 6), more informative than any single figure drawn from it.

3 Layout search

3.1 Background and scope

A companion paper [1] establishes that Qiskit’s `VF2Layout` pass fails—reporting no solution where a valid embedding exists—on a specific, reproducible class of saturated layout instances, and that a single `rustworkx.vf2_mapping` call with a fixed node ordering (`id_order=True`) resolves every tested instance of that class in microseconds. This paper does not re-derive that finding; it reports the engineering built on top of it and its independent, end-to-end verification.

PSF-Zero’s own layout prototype, `smart_vf2_layout`, predates that finding and originally used a different strategy: a sequence of BFS-family node orderings, tried with `id_order=True`, falling back to `rustworkx`’s own heuristic ordering if all failed. We found and fixed four defects in this prototype in the course of incorporating the companion paper’s finding, summarized in Table 2 and detailed in Section 5.

Defect	Fix	Effect
Natural ordering (matching the companion paper’s finding) never attempted	Added as the first candidate ordering	56.8× median speedup on the search step
Feasibility guard compared interaction-graph edge count to device matching size, valid only for the prototype’s original circuit family	Compares the interaction graph’s own maximum matching instead	Every chain-shaped circuit family used in this project reaches the search at all for the first time
Guard recomputed the device’s own matching on every call	Cached per device, keyed on graph structure	52.4% reduction in guard cost
Ordering candidates computed eagerly regardless of use	Generator-based, lazy evaluation	Modest additional saving; the larger residual cost was traced to graph construction, not ordering

Table 2: Defects found and fixed in `smart_vf2_layout`.

3.2 Is the feasibility guard worth its cost?

The repaired guard adds a fixed per-call cost. We measured directly whether this is worthwhile, using instances specifically constructed to be infeasible by matching capacity but not by raw qubit count—the only case in which the guard can add value, since qubit-count infeasibility is caught by the search itself immediately regardless.

Excluding the control case (where the guard is not the relevant mechanism), the guard breaks even against the unguarded search at roughly one infeasible call in 482, using the smallest, fully-concluded search time in Table 3 as a conservative bound.

Instance	Guard rejection	Unguarded search
Matching-imbalanced, small	0.70 ms	1,642.5 ms
Matching-imbalanced, larger	1.22 ms	$\geq 2,000.5$ ms (budget-limited)
Matching-imbalanced, off-by-one	1.18 ms	$\geq 2,000.1$ ms (budget-limited)
Control (also infeasible by qubit count)	1.00 ms	0.30 ms

Table 3: Guard cost versus unguarded search cost on constructed infeasible instances. Two of four searches did not conclude within the 2-second budget used and are reported as lower bounds. The control row confirms the guard is not simply always cheaper—on an instance the search itself rejects immediately, the guard’s own fixed cost exceeds it.

3.3 End-to-end verification

We verified the repaired search through `compile_for_hardware(layout_search=True)`, the system’s own public entry point, rather than through `smart_vf2_layout` alone, on saturated instances at two grid sizes (6×7 , 8×8) in two circuit families: the prototype’s original disjoint-pair family, and a chain-shaped family the pre-fix guard rejected outright.

Coverage. All 26 tested instances (matching the companion paper’s own primary instance family) find a valid layout through this entry point, versus 0 of 26 before the guard fix—since the guard previously rejected every one before any search ran.

Structural validity. Every routed output was checked directly against the coupling map’s own edge set. Across all measured configurations, coupling-map violations: 0.

Unitary equivalence. At $n = 6$ (both circuit families, three arms, three seeds, two repeats: 36 runs), we compared each output’s operator against the input’s, correcting for the routing-induced qubit permutation using Qiskit’s own `TranspileLayout` interface. The permutation-handling logic itself was independently verified against hand-constructed test cases before use, since an error there could produce a false pass or false fail indistinguishable from a genuine result. Minimum fidelity across all 36 runs: 0.999999999999994—at the floating-point noise floor, not near any threshold.

Practical benefit. On the chain-shaped family specifically (the instances the guard fix newly unlocks), the repaired search produces a modest but consistent end-to-end improvement over the default layout path: $1.07\times$ faster at both grid sizes tested, never worse. We report this without embellishment: it is a real but small effect, not the multiple-orders-of-magnitude figure that applies to the isolated search step in Table 2, because the search is a small fraction of the full compilation pipeline’s own cost.

4 Generalization

Every result in Section 3 used square grids. We tested the repaired search on two structurally different topologies: **brick** (a lattice on which the prototype’s own earlier BFS orderings had previously found nothing on saturated instances) and heavy-hex (bipartite-imbalanced, so qubit-count saturation is unattainable; we use saturation relative to the lattice’s own maximum matching instead).

The **brick** result is, to us, the most striking evidence that this is not a square-grid-specific fix: the repaired search succeeds in about a millisecond on instances where the prototype’s entire prior strategy—every BFS-family ordering it previously used, exhausted—found nothing at all.

Topology	n	Repaired search	Prior orderings
brick, 6×7	42	found, 1.25 ms	none found, 875 ms
brick, 8×8	64	found, 1.08 ms	none found, 996 ms
heavy-hex, $d=3$	19	found, 0.25 ms	—
heavy-hex, $d=5$	57	found, 0.69 ms	—

Table 4: Repaired search versus the prototype’s own prior ordering strategies, on topologies neither was developed against.

5 Defects found

We report every defect found during this investigation, including two that reflect on our own prior conclusions rather than only on the system under test.

The four layout-search defects are summarized in Table 2 above.

A root-cause mischaracterization, corrected. An early stage of this investigation attributed part of the guard’s inefficiency to a specific code pattern (`find_bit`-based qubit resolution) found in an intermediate development revision. Re-reading the actual production codebase directly showed that revision was never deployed; the pattern in question does not appear in the code this paper evaluates. We retract the earlier attribution rather than let it stand alongside the corrected account.

A cumulative-speedup figure, corrected. Table 1 reports the archive session’s own speedup as $7.00\times$; our own earlier project record reported $8.87\times$ for the same session. We recomputed directly from that session’s own raw per-call data while preparing this paper and found the discrepancy; we do not know whether the earlier figure was a transcription error or drawn from a different run than the archived raw data represents, and we do not guess between these. We use the recomputed, independently-verifiable figure throughout this paper.

A memory-growth question, resolved. Disabling Python’s garbage collector during repeated `compile()` calls revealed measurable memory growth. We traced this to two independent, distinct phenomena: reference cycles in Qiskit’s own circuit objects (not PSF-Zero’s own code), reclaimed cleanly and at a bounded, steady rate under normal (garbage-collection-enabled) operation; and a separate, one-time interpreter warm-up cost in generic runtime objects, confirmed by extending the measurement across multiple checkpoints within a single process and observing growth reach exactly zero after the first several hundred calls. Neither is a genuine leak.

6 An unresolved finding

Table 1’s standard-deviation column shows a $34\text{--}54\times$ reduction between the archive session and the two later sessions (confirmed by Levene’s test, $p \approx 0$ for both comparisons against the archive; the two later sessions do not differ significantly from each other for the `verify=True` arm, $p = 0.082$). We investigated this directly rather than let it pass unremarked.

We obtained the actual source code from both sessions and examined the diff line by line. Two changes present between the sessions were traced as candidate explanations and ruled out: a change to the gate synthesizer’s own internal caching eviction policy does not plausibly matter for this benchmark, because the benchmark generates a fresh, uniquely-seeded random circuit every iteration, so the cache provides negligible hit-rate benefit under either eviction policy; and a change eliminating redundant verification work applies only to a different code path (`verify="strict"`) not exercised by this benchmark. The compiled Rust core’s own decomposition routine is unchanged

between sessions in every way that could plausibly affect per-call timing. We found no other candidate in the diff.

We did not investigate two remaining candidate explanations: differences in the versions of dependency packages installed on the measurement machine between sessions (not tracked at the time), and machine or operating-system-level environmental factors. We report this as a closed but unresolved investigation rather than speculate further.

7 Update, 22–26 September 2026 (version 2)

This section reports work done after version 1. Every item was pre-registered before measurement, in the project’s chronological record (Addenda 181–197 of the *spare-qubit-cliff* series), where the full predictions, logs and raw data are kept. All measurements use Qiskit 2.5.2 on one workstation (WSL2, 12 cores); the revisions of `psf_compile.py` are named by their `VERSION` string.

7.1 A defect inherited from Qiskit’s CX-basis synthesis

Qiskit’s `TwoQubitBasisDecomposer(CXGate(), euler_basis="ZSX")` returns an incorrect circuit for a narrow band of inputs. For the canonical gate $\exp(i(aXX + bYY + cZZ))$ with $a = 0.6$, $b = 0.3$, the emitted three-CX circuit has average gate infidelity 6.99×10^{-2} for $c = 3 \times 10^{-8}$, 10^{-7} , 2×10^{-7} and 3×10^{-7} , and is exact at $c = 10^{-8}$ and from $c = 5 \times 10^{-7}$ upward. The same decomposer with its default Euler basis is exact on the same inputs. The failure also appears in plain `transpile()` of a `UnitaryGate` to `basis_gates=["cx", "rz", "sx", "x"]` at every optimization level (0–3), with the same infidelity, and with single-qubit layers on either side of the core. It did not appear with a CZ basis, with a two-qubit backend target carrying error data, or when the same gate was given as RXX/RYY/RZZ gates. The constant infidelity across the band suggests a single wrongly taken branch rather than accumulated rounding; this is an inference from outside, as we did not inspect Qiskit’s source. A minimal reproduction using only Qiskit is in the repository; at the time of writing it had not yet been reported upstream, and we did not find an existing report of it.

Effect on PSF-Zero. Every revision up to and including the one evaluated in version 1 re-expressed the entangling core through this decomposer on the `cx` path, and also used it for whole blocks the Rust core declines as degenerate. Such blocks were emitted unchecked. The band is reached only when one canonical coordinate of a block lies within roughly 3×10^{-8} – 3×10^{-7} of a multiple of $\pi/2$; the random dense-pair blocks of every measurement in version 1 fall there with negligible probability, and the exact $n = 6$ comparisons of Section 2 and Section 3 confirm that none of those outputs was affected. Circuits with small two-qubit interaction angles can reach the band.

Fix. In the current revision (2026-09-26.4) every circuit obtained from Qiskit’s decomposer is checked: its unitary is computed and accepted only if its average gate infidelity is at most 10^{-8} (ten times the decomposer’s own default requested fidelity, so its documented approximations still pass), with its global phase corrected to match exactly. Otherwise the default-Euler decomposer is tried, and for a canonical core the closed form of Section 7.3 is used as a last resort; a whole block that fails both raises an error rather than being emitted. On 272 near-degenerate blocks in four families, with the check disabled 48 blocks have infidelity 6.99×10^{-2} ; with it enabled the worst is 1.6×10^{-12} , the default-Euler retry succeeded every time it was needed, and no CX count changed where the unchecked result had been correct.

7.2 Precision under repeated recompilation

In a variational or feedback loop the compiled circuit is recompiled many times, and each lap’s output is the next lap’s input. We measured the largest per-pair deviation from the original circuit over ten such laps on a 120-qubit device model (FakeNighthawk). Table 5 summarizes the revisions. The first fix (2026-09-26.2) polishes the Rust core’s 16 decomposition parameters with a closed-form Gauss–Newton step before the circuit is built; the core’s own output had been off by up to $\sim 10^{-11}$ near the Weyl-chamber face $c = 0$. The later revisions change how the entangling core is emitted (Section 7.3).

Revision	worst per-block error	drift, lap 1	drift, lap 10
2026-09-21 (version 1 era)	1.76×10^{-11}	1.76×10^{-11}	6.47×10^{-10}
2026-09-26.2 (polishing)	6.2×10^{-14}	6.2×10^{-14}	7.27×10^{-13}
2026-09-26.3 (closed form, first form)	6.2×10^{-14}	2.32×10^{-13}	1.92×10^{-12}
2026-09-26.4 (current)	6.2×10^{-14}	6.21×10^{-14}	6.21×10^{-13}
Qiskit <code>optimization_level=3</code>	1.8×10^{-13}	3.37×10^{-13}	3.40×10^{-13}

Table 5: Worst per-block synthesis error on 60 random blocks, and the largest per-pair deviation after 1 and 10 recompilation laps (FakeNighthawk, 120 logical qubits). Qiskit’s error does not grow across laps; PSF-Zero’s grows exactly linearly (lap k : $k \times 6.21 \times 10^{-14}$), so it exceeds Qiskit’s after six laps and 10^{-12} after about 16 laps.

7.3 Compile time on a current IBM topology

FakeNighthawk models IBM’s 120-qubit square-lattice generation. Its coupling graph has a perfect matching, so a circuit of 60 disjoint interacting pairs saturates it—the situation in which the companion paper’s failure region occurs [1]. On such circuits (dense pair blocks, 20 random two-qubit gates per pair), Qiskit’s `transpile(optimization_level=3)` takes 12.7–13.0 s. A per-pass breakdown shows two searches of nearly equal cost: `VF2Layout`, which reports no solution after 6.2–6.3 s, and `VF2PostLayout`, which runs after routing for another 6.4–6.7 s. With eight spare qubits the same compile takes 0.155 s.

A per-stage breakdown of PSF-Zero on the same circuits showed that the Rust core accounts for about 1% of its time, and led to three changes: (i) when the interaction graph is a set of disjoint pairs, the layout is read directly from a maximum matching of the coupling graph—placing k disjoint pairs is exactly choosing k disjoint physical edges—so no subgraph search runs (15.3 ms $\rightarrow$ 0.45 ms for the layout step); (ii) the entangling core is emitted in closed form as three CXs (the Vatan–Williams construction [4]), with the rotations in its two middle gaps written directly as one `sx` each, instead of being decomposed a second time by Qiskit; (iii) the check of Section 7.1. The median compile time on the saturated circuits fell from 83 ms (2026-09-26.2) to 49.8 ms, with two-qubit count (180), `sx` count (840) and exact per-pair equivalence unchanged and depth reduced from 23 to 21.

We state the comparison with Qiskit carefully. The ratio on the saturated circuits is about 260 $\times$, but roughly half of Qiskit’s time is `VF2PostLayout`, which PSF-Zero skips because it supplies an initial layout. That pass rescores a layout against the device’s error rates; here it finds nothing better, but on hardware skipping it can cost fidelity. Section 7.4 addresses that cost directly. Off the failure region (eight spare qubits) PSF-Zero is about 3 $\times$ faster.

7.4 An error-aware layout, evaluated on a model

The matching of Section 7.3 can be weighted. With edge weight $n_2 \log(1 - e_{pq}) + n_1(\log(1 - e_p) + \log(1 - e_q))$ —two-qubit gate error e_{pq} on the edge, **sx** error e_p, e_q on its endpoints, n_2 and n_1 the expected gate counts per pair and per qubit—a maximum-weight matching maximizes the product of the fidelities of the gates the circuit will use, exactly, since each matched edge covers exactly its two qubits. We compared its estimated success probability (ESP, the product of $1 - \text{error}$ over every emitted instruction, from the backend’s error data) with Qiskit `optimization_level=3`, whose `VF2PostLayout` scores layouts with error data of its own.

Device model (logical qubits)	PSF-Zero, weighted	Qiskit L3	ratio
FakeNighthawk (120)	−1800.253	−1800.341	1.22×
FakeNighthawk (112)	−0.200	−0.299	1.26×
FakeTorino (80)	−0.226	−0.550	2.11×
FakeFez (80)	−0.211	−0.321	1.29×

Table 6: $\log_{10}$ ESP on dense-pair-block circuits. Two-qubit and **sx** counts are identical between the arms in every row, so the differences are due to layout alone. The FakeNighthawk model contains single-qubit gates with error 1.0; at 120 logical qubits every qubit is used and both arms pay for them (hence −1800).

The weighted layout reaches the higher ESP on all four models, at PSF-Zero’s compile times of 41–53 ms against Qiskit’s 0.16–0.26 s off the failure region and 12.9 s on it. A first version that weighted two-qubit errors only lost on FakeNighthawk (112) by placing pairs on a qubit whose **sx** has error 1.0; adding single-qubit errors, as anticipated in its pre-registration, removed the loss. These are model results: ESP assumes independent gate errors; FakeNighthawk’s error values are synthetic by its own description and FakeTorino’s and FakeFez’s are dated calibration snapshots; $n_1 = 2n_2 + 1$ is a heuristic tuned to three-CX blocks; and the method applies only to circuits whose interaction graph is a set of disjoint pairs. Whether the difference survives on hardware is untested.

8 Related work

Cartan/KAK decomposition of two-qubit unitaries is well-established [2, 3]; our contribution is not the decomposition itself but a compiled, production-oriented implementation and its integration into a verified compilation pipeline. The layout search builds directly on the ordering-sensitivity finding of our companion paper [1]; SABRE [5] remains the relevant point of comparison for heuristic routing when no perfect layout exists.

9 Conclusion

PSF-Zero combines an analytic gate synthesizer with a layout search engineered around a specific, externally-motivated ordering insight, and we have verified both components more thoroughly than is typical for a research prototype: exact correctness at small scale, structural validity and unitary equivalence end-to-end, and generalization beyond the topology used in development. We have also been direct about what remains unresolved—an unexplained variance reduction in our own timing measurements—and about two mistakes in our own prior reporting that we found and corrected while preparing this paper. We consider this standard of self-correction a necessary part of the contribution, not a caveat to it.

(*Version 2.*) The same standard applied after version 1 found a defect whose source is outside PSF-Zero—in Qiskit’s CX-basis synthesis—but whose effect PSF-Zero had inherited silently. It is now checked for on every affected block. The current revision compiles the saturated 120-qubit case in under 50 ms, keeps repeated-recompilation drift below 10^{-12} for about sixteen laps, and, on device models, places pairs with a higher estimated success probability than Qiskit’s most thorough preset. Each of these is reported with its limits in Section 7.

Data and code availability

All raw measurement data, defect reports, and the complete chronological verification record, including every correction discussed in Section 5, are available at <https://github.com/TN-Holdings-LLC/psf-zero>. The work of Section 7 is recorded in Addenda 181–197 of `docs/findings/spare-qubit-cliff-combined`; the minimal reproduction of the Qiskit defect is `benchmarks/repro_qiskit_zsx_2q_v2.py`.

References

- [1] TN Holdings, *Ordering Sensitivity in Subgraph-Isomorphism Layout Search: A Characterization of Catastrophic Failure Regions in Quantum Circuit Transpilation*, 2026.
- [2] N. Khaneja and S. J. Glaser, *Cartan decomposition of $SU(2^n)$ and control of spin systems*, Chemical Physics, 267(1–3):11–23, 2001.
- [3] R. R. Tucci, *An introduction to Cartan’s KAK decomposition for QC programmers*, arXiv:quant-ph/0507171, 2005.
- [4] F. Vatan and C. Williams, *Optimal quantum circuits for general two-qubit gates*, Physical Review A, 69:032315, 2004.
- [5] G. Li, Y. Ding, and Y. Xie, *Tackling the qubit mapping problem for NISQ-era quantum devices*, ASPLOS ’19, pp. 1001–1014, 2019.